Conocimiento y Razonamiento Automatizado

PECL3

Scheme - Listas

Grado en Ingeniería Informática
Universidad de Alcalá

Álvaro Fernández Narciso
Andrea Pascual Aguilera
Sergio Alonso Zarcero

Mayo 2024-2025

Índice

Introducción *3*

Reparto de tareas *3*

Implementación de listas *3*

Creación de listas 3

Visualización de listas 4

Conversión entre Church y decimal 4

Extracción de elementos 5

Comprobación de lista vacía 5

Desarrollo *5*

Suma de elementos de una lista 5

Resta de elementos de una lista 5

Cálculo del máximo y mínimo 6

Grado de cumplimiento *6*

Conclusión *7*

Fuentes consultadas *7*

Introducción

El objetivo de esta práctica es implementar listas de números enteros utilizando λ-cálculo en Scheme, basándonos en la codificación de enteros proporcionada en el archivo enteros.rkt[1]. La práctica consiste en desarrollar tres operaciones sobre listas de enteros: suma de elementos, resta de elementos entre dos listas, y cálculo del máximo y mínimo de una lista. Este informe detalla el proceso de implementación, el reparto de tareas, el grado de cumplimiento de los requisitos, y las fuentes consultadas.

Reparto de tareas

Para el reparto de tareas, Sergio se ha encargado de la base de la práctica realizando las funciones base. Ha desarrollado las funciones auxiliares, como el manejo de listas y conversiones, así como las funciones de impresión.

Andrea se ha encargado de la implementación de la suma y la resta de los elementos de la lista, así como de la comprobación de su funcionamiento.

Álvaro se ha encargado de la implementación del cálculo del máximo y del mínimo de los elementos de la lista y de su comprobación.

Repartiendo el trabajo de esta manera, conseguimos minimizar las dependencias, agrupamos las funciones relacionadas lógicamente y facilitamos la colaboración.

Implementación de listas

Las listas se implementan utilizando la codificación de Church en λ-cálculo, representándolas como estructuras recursivas basadas en pares ordenados. A continuación, se detallan los aspectos clave de su creación, manipulación, visualización y conversión numérica.

Creación de listas

[1] Para poder importar el archivo se ha escrito #lang racket y (provide (all-defined-out)) al comienzo de este archivo.

Las listas se construyen mediante dos funciones principales:

Lista vacía (emptyList): Representa una lista sin elementos, definida como una función que devuelve su argumento.

Construcción de listas (makeList): Crea una lista añadiendo un elemento (cabeza) a una cola existente, codificada como un par (false, (elemento, cola))

Ejemplo: La lista [4,−2, 7] se construye como:

```
 ( define listX (( makeList four )
 (( makeList minusTwo )
(( makeList seven ) emptyList ) ) ) )
```

Visualización de listas

La función printList muestra los elementos de una lista en formato legible, como [4,−2, 7, nil]. Utiliza recursión con el combinador de punto fijo Y y convierte cada elemento a decimal con churchToInteger.

Si la lista es vacía, imprime nil]. Si no, imprime la cabeza, una coma, y procesa la cola recursivamente.

Conversión entre Church y decimal

Los números en codificación de Church se convierten a formato decimal para su visualización:

De Church a natural (churchToNatural): Convierte un número natural de Church (que aplica una función n veces) a decimal aplicando add1 desde 0

De Church a entero (churchToInteger): Convierte un entero codificado como (m, n) (representando m− n) a decimal, considerando si es positivo o negativo.

Se crea un caso especial para el cero ya que no es ni positivo ni negativo, causando conflicto si este último aspecto no es implementado.

Extracción de elementos

Se definieron funciones para acceder a los componentes de una lista:
Cabeza (head): Extrae el primer elemento de una lista no vacía:
Cola (tail): Devuelve la sublista sin la cabeza
Ejemplo: Para [4,−2, 7], head devuelve four, y tail devuelve [−2, 7].

Comprobación de lista vacía

La función isEmpty verifica si una lista es vacía.
Una lista no vacía está codificada como (false, (elemento, cola)), devolviendo false.
La lista vacía devuelve true, permitiendo controlar la recursión en otras funciones.

Desarrollo

La implementación se realizó en el archivo main.rkt, utilizando la codificación de listas en λ-cálculo y las operaciones sobre enteros definidas en enteros.rkt. A continuación, se describen las principales funciones implementadas:

Suma de elementos de una lista

La función sumListElements calcula la suma de todos los elementos de una lista de enteros, retornando 0 si la lista está vacía. Utiliza recursión a través del combinador de punto fijo Y , sumando la cabeza de la lista con la suma de la cola mediante la función sument de enteros.rkt.

Resta de elementos de una lista

La función resListElements toma como ejemplo sumListElements, con un sutil cambio en el resultado a retornar. Esto se debe a que, en vez de tratarse de una

suma recursiva de elementos, se le resta la suma recursiva de la cola a la cabeza, obteniendo así el resultado esperado. La resta secuencial implementada se consigue mediante restaent definida en enteros.rkt.

Cálculo del máximo y mínimo

Las funciones maxElement y minElement encuentran el elemento máximo y mínimo de una lista, respectivamente, inicializando el valor actual con la cabeza de la lista. Utilizan las funciones greater y less para comparar elementos, y recursión para procesar la cola de la lista.

Grado de cumplimiento

Todos los requisitos de la práctica han sido implementados satisfactoriamente:
Suma de elementos: La función sumListElements calcula correctamente la suma de los elementos de una lista.
Resta de listas: La función resListElements realiza la resta secuencial de los elementos de una lista.
Máximo y mínimo: Las funciones maxElement y minElement encuentran correctamente el máximo y mínimo de una lista.
No se han detectado errores en la ejecución de las funciones, y las pruebas con las listas de ejemplo (listX y listY) producen los resultados esperados.

```
EJECUCION DE PRUEBA
La lista X es: [4, -2, 7, -1, 0, -5, nil]

La suma de los elementos de la lista X es: 3
La resta de los elementos de la lista X es: 5
El máximo de la lista X es: 7
El mínimo de la lista X es: -5

La lista Y es: [-3, 8, -6, 2, -4, 1, nil]

La suma de los elementos de la lista Y es: -2
La resta de los elementos de la lista Y es: -4
El máximo de la lista Y es: 8
El mínimo de la lista Y es: -6
```

Conclusión

La práctica ha sido completada con éxito, implementando todas las operaciones solicitadas sobre listas de enteros en Scheme. El uso del λ-cálculo y el combinador de punto fijo permitió desarrollar funciones recursivas que cumplen con los requisitos. El trabajo en equipo fue eficiente, con un reparto claro de tareas y una colaboración efectiva en la redacción del informe.

Fuentes consultadas

Para el desarrollo de esta práctica se han consultado diversas fuentes, tanto proporcionadas directamente en la asignatura como externas, con el fin de comprender, implementar y validar correctamente las estructuras y operaciones requeridas sobre listas de enteros mediante λ-cálculo en el lenguaje Scheme.

1. **Materiales de la asignatura:**

- **Apuntes teóricos del curso ("cra_notes.pdf") [1]**
  Hemos utilizado especialmente los capítulos dedicados al λ-cálculo (capítulo 4), que abordan en detalle los λ-términos, estructuras de datos codificadas (booleanos, pares, números de Church), listas y la definición del combinador de punto fijo (secciones 4.3 y 4.4). Estas secciones nos han sido clave para codificar las listas funcionales y definir operaciones recursivas.

- **Enunciado oficial de la Práctica 3 - CRA ("Práctica 3_CRA.2025.pdf") [2]**
  Este documento nos ha guiado en la implementación de las funciones solicitadas y ha ofrecido pautas importantes para trabajar de forma funcional y currificada, incluyendo una implementación válida del combinador de punto fijo Y para Scheme. En el propio enunciado se incluye también un enlace de referencia a un recurso web clave [3].

- **Archivo `enteros.rkt` [3]**
  Proporcionado por el profesor, con la implementación completa de los enteros mediante pares ordenados y su aritmética (suma, resta, multiplicación, división, MCD, comparación, valor absoluto, y normalización a representantes canónicos). Este archivo ha sido la base sobre la que hemos construido toda la lógica de las listas.

- **Archivos de ejemplo proporcionados**
  - `ejemplo_adicional_1.rkt,`
  - `ejemplo_adicional_2_sumar_lista.rkt`
  - `ejemplo_adicional_2_sumar_lista.rkt`
  - `ejemplo_adicional_4_funciones_lambda.rkt`

  Nos han servido de apoyo conceptual para comprender la estructura de pares en Scheme, el manejo de listas tradicionales y la utilización básica de funciones lambda.

2. Documentación técnica y bibliografía externa:

- *Lambda Calculus in Scheme*, por Shlomi Fish [4]. Utilizado para implementar el combinador de punto fijo Y.
- *The Racket Documentation* [5]: guía oficial de Racket para sintaxis, funciones y estructuras funcionales. Incluye *The Racket Guide* y *The Racket Reference Manual*.
- *Scheme Wiki* [6]: documentación general sobre el lenguaje Scheme y sus variantes.
- *Structure and Interpretation of Computer Programs (SICP)* [7]: obra de referencia en programación funcional.
- *The Lambda Calculus: Its Syntax and Semantics* [8]: texto teórico clásico sobre el λ-cálculo y su semántica.
- *Stack Overflow* [9]: fuente auxiliar para resolver dudas puntuales sobre recursividad y sintaxis funcional en Scheme.
- *Repositorios educativos en GitHub*: utilizados para consulta de ejemplos de codificación funcional, sin copiar contenido directamente.

A continuación, se citan todas las fuentes consultadas:

[1] D. Castro y J. E. Morais, *Apuntes de Conocimiento y Razonamiento Automatizado*, Universidad de Alcalá, Curso 2021–2022.

[2] Universidad de Alcalá, *Práctica 3 – Conocimiento y Razonamiento Automatizado*, CRA, Curso 2024–2025.

[3] *Archivo enteros.rkt*, Universidad de Alcalá, 2025.

[4] S. Fish, *Lambda Calculus in Scheme*. [En línea]. Disponible en: https://www.shlomifish.org/lecture/Lambda-Calculus/slides/lc_Y.scm.html

[5] The Racket Project, *Racket Documentation*. [En línea]. Disponible en: https://docs.racket-lang.org/

[6] Scheme Language Steering Committee, *Schemers.org*. [En línea]. Disponible en: https://schemers.org/

[7] H. Abelson, G. J. Sussman y J. Sussman, *Structure and Interpretation of Computer Programs*, 2ª ed., Cambridge, MA: MIT Press, 1996.

[8] H. P. Barendregt, *The Lambda Calculus: Its Syntax and Semantics*, 2ª ed., Amsterdam: North-Holland, 1984.

[9] Stack Overflow. [En línea]. Disponible en: https://stackoverflow.com